

ADVERSARIAL GAMES



ADVERSARIAL GAMES

Grid Knockout

Player 2 has \$10. Player 1 is playing for a share of Player 2's money. Each turn the players cross out half of the remaining grid. Player 1 can cross out the left or right half. Player 2 can cross out the top or bottom half. The number left at the end is the amount of money Player 1 gets from Player 2.

ADVERSARIAL GAMES

Grid Knockout

Player 2 has \$10. Player 1 is playing for a share of Player 2's money. Each turn the players cross out half of the remaining grid. Player 1 can cross out the left or right half. Player 2 can cross out the top or bottom half. The number left at the end is the amount of money Player 1 gets from Player 2.

7	7	3	5
5	3	2	4
6	3	3	7
2	3	7	8

ADVERSARIAL GAMES

Grid Knockout

Player 2 has \$10. Player 1 is playing for a share of Player 2's money. Each turn the players cross out half of the remaining grid. Player 1 can cross out the left or right half. Player 2 can cross out the top or bottom half. The number left at the end is the amount of money Player 1 gets from Player 2.

7	7		
5	3		
6	3		
2	3		

ADVERSARIAL GAMES

Grid Knockout

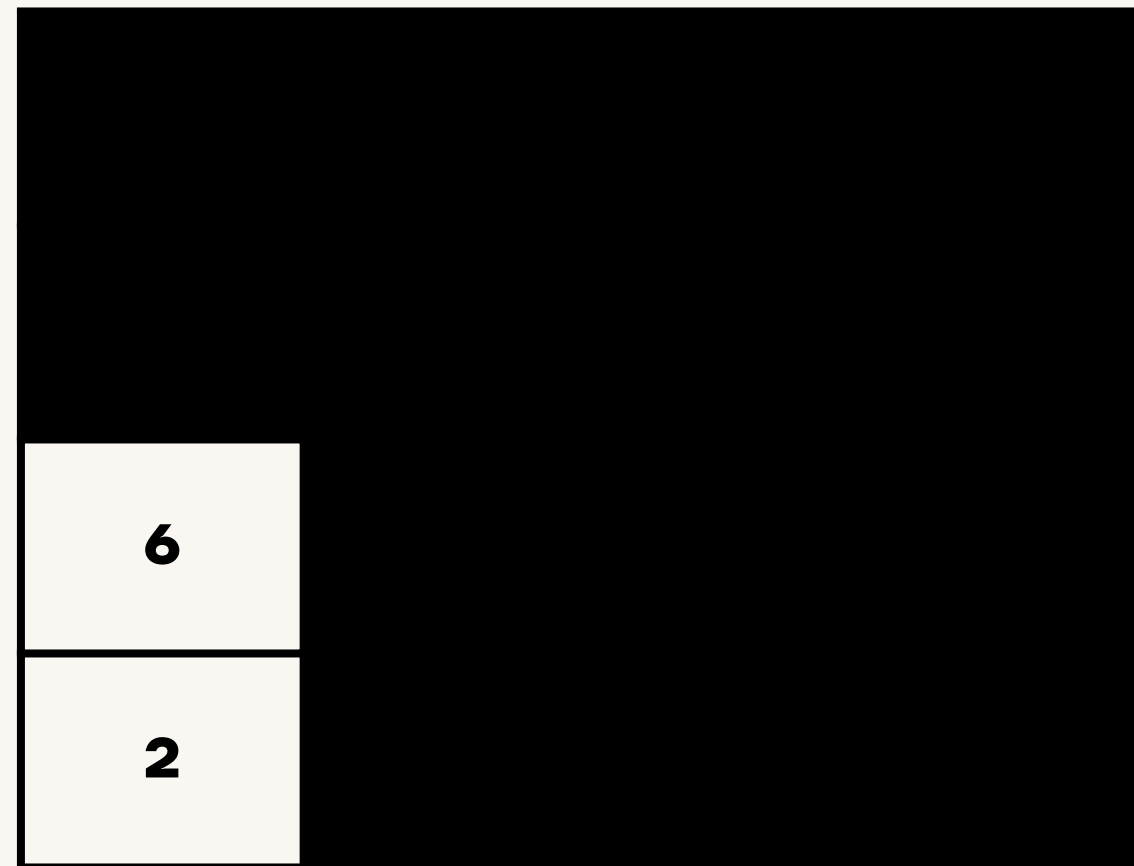
Player 2 has \$10. Player 1 is playing for a share of Player 2's money. Each turn the players cross out half of the remaining grid. Player 1 can cross out the left or right half. Player 2 can cross out the top or bottom half. The number left at the end is the amount of money Player 1 gets from Player 2.

6	3		
2	3		

ADVERSARIAL GAMES

Grid Knockout

Player 2 has \$10. Player 1 is playing for a share of Player 2's money. Each turn the players cross out half of the remaining grid. Player 1 can cross out the left or right half. Player 2 can cross out the top or bottom half. The number left at the end is the amount of money Player 1 gets from Player 2.



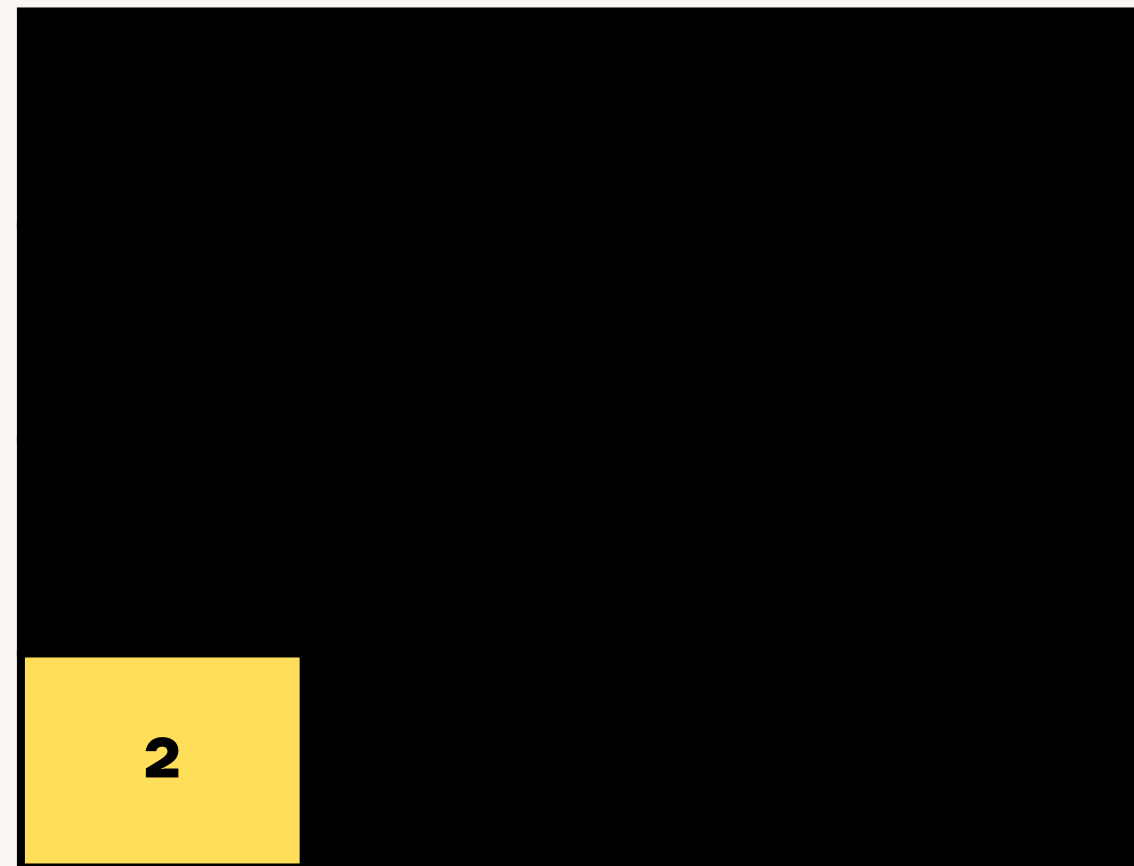
ADVERSARIAL GAMES

Grid Knockout

Min

Max

~~Player 2~~ has \$10. ~~Player 1~~ is playing for a share of Player 2's money. Each turn the players cross out half of the remaining grid. Player 1 can cross out the left or right half. Player 2 can cross out the top or bottom half. The number left at the end is the amount of money Player 1 gets from Player 2.



ADVERSARIAL GAMES

Review:

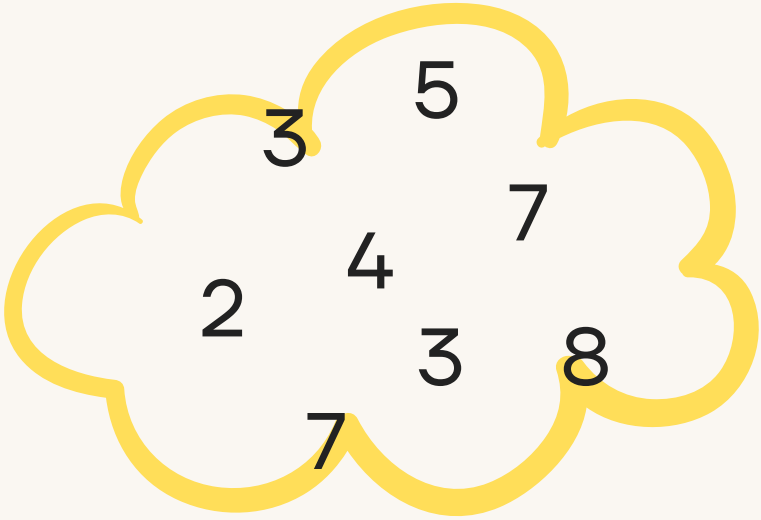
What was your general strategy when it was your turn?

What factors did you consider before crossing out half the grid?

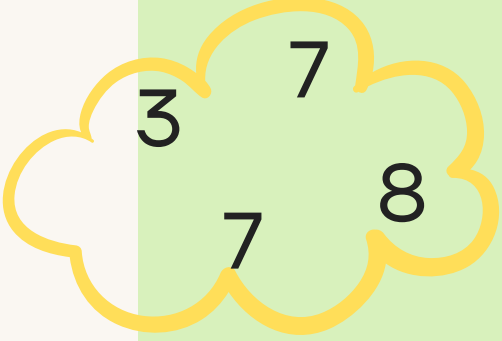
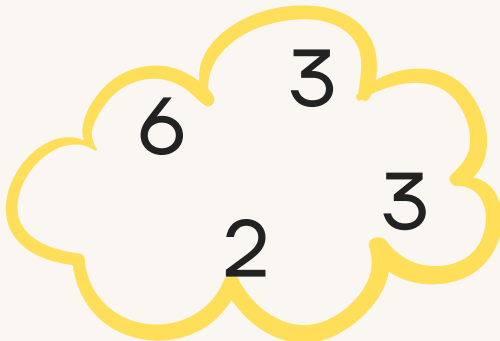
How did you try to anticipate what the other player would do next?

7	7	3	5
5	3	2	4
6	3	3	7
2	3	7	8

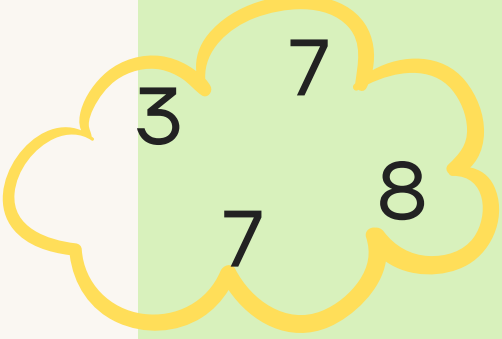
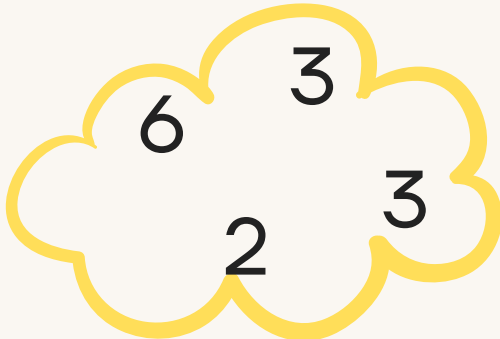
7	7	3	5
5	3	2	4
6	3	3	7
2	3	7	8



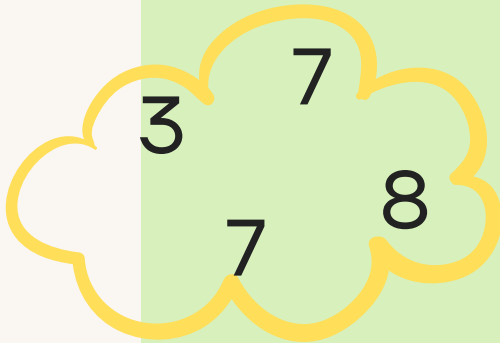
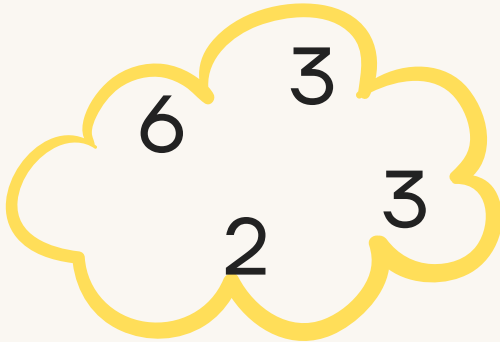
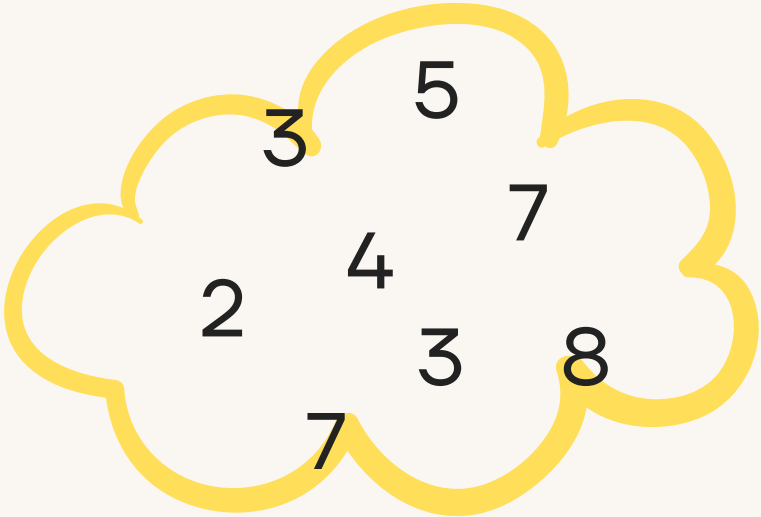
7	7	3	5
5	3	2	4
6	3	3	7
2	3	7	8



7	7	3	5
5	3	2	4
6	3	3	7
2	3	7	8



7	7	3	5
5	3	2	4
6	3	3	7
2	3	7	8



7

5

7

3

6

2

3

3

3

2

5

4

3

7

7

8

MINIMAX

AI for fully observable, zero-sum games (ex. DeepBlue, AlphaGo)

Adversarial search for the best action when both players play optimally

function MINIMAX-SEARCH(*game, state*) **returns** *an action*

$\text{player} \leftarrow \text{game.TO-MOVE}(\text{state})$

$\text{value, move} \leftarrow \text{MAX-VALUE}(\text{game, state})$

return *move*

function MAX-VALUE(*game, state*) **returns** *a (utility, move) pair*

if *game.IS-TERMINAL(state)* **then return** *game.UTILITY(state, player), null*

$v \leftarrow -\infty$

for each *a in game.ACTIONS(state)* **do**

$v2, a2 \leftarrow \text{MIN-VALUE}(\text{game, game.RESULT}(\text{state, a}))$

if $v2 > v$ **then**

$v, \text{move} \leftarrow v2, a$

return *v, move*

7	7	3	5
5	3	2	4
6	3	3	7
2	3	7	8

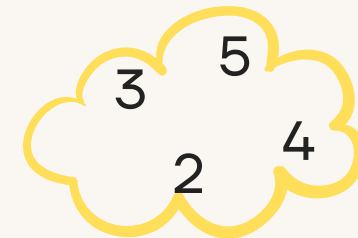
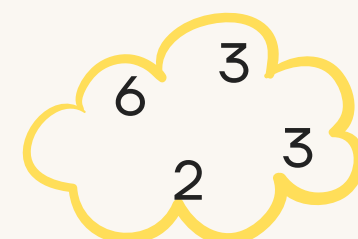


Max

Min

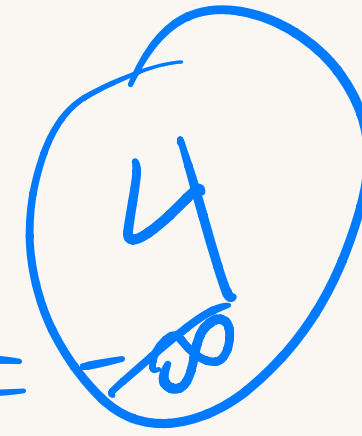
Max

Min



7 5 7 3 6 2 3 3 3 2 5 4 3 7 7 8

7	7	3	5
5	3	2	4
6	3	3	7
2	3	7	8



Max

$v = -\infty$ 3

$v = -\infty$ 4

Min

$v = -\infty$ 6

$v = -\infty$ 3

$v = -\infty$ 4

$v = -\infty$ 7

Max

$v = -\infty$ 5

$v = -\infty$ 3

$v = -\infty$ 2

$v = -\infty$ 3

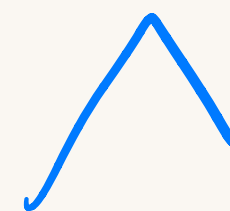
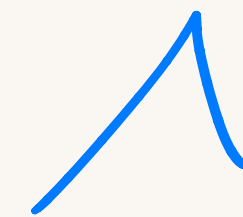
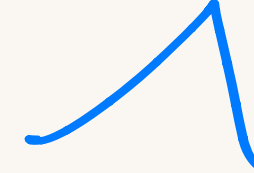
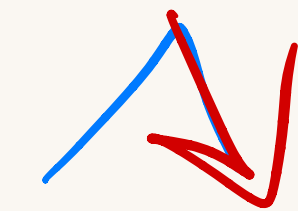
$v = -\infty$ 2

$v = -\infty$ 4

$v = -\infty$ 3

$v = -\infty$ 7

Min



7 5 7 3 6 2 3 3 3 2 5 4 3 7 7 8

4

max



4

3

min



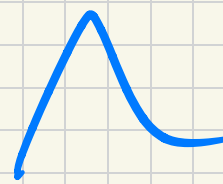
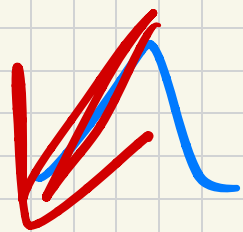
4

6

8

3

max



4

3

6

3

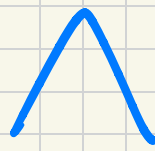
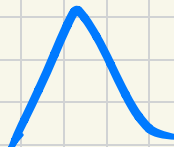
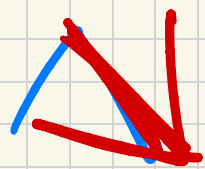
8

5

0

3

min



7

4

5

3

6

7

3

7

9

8

5

9

3

0

9

3

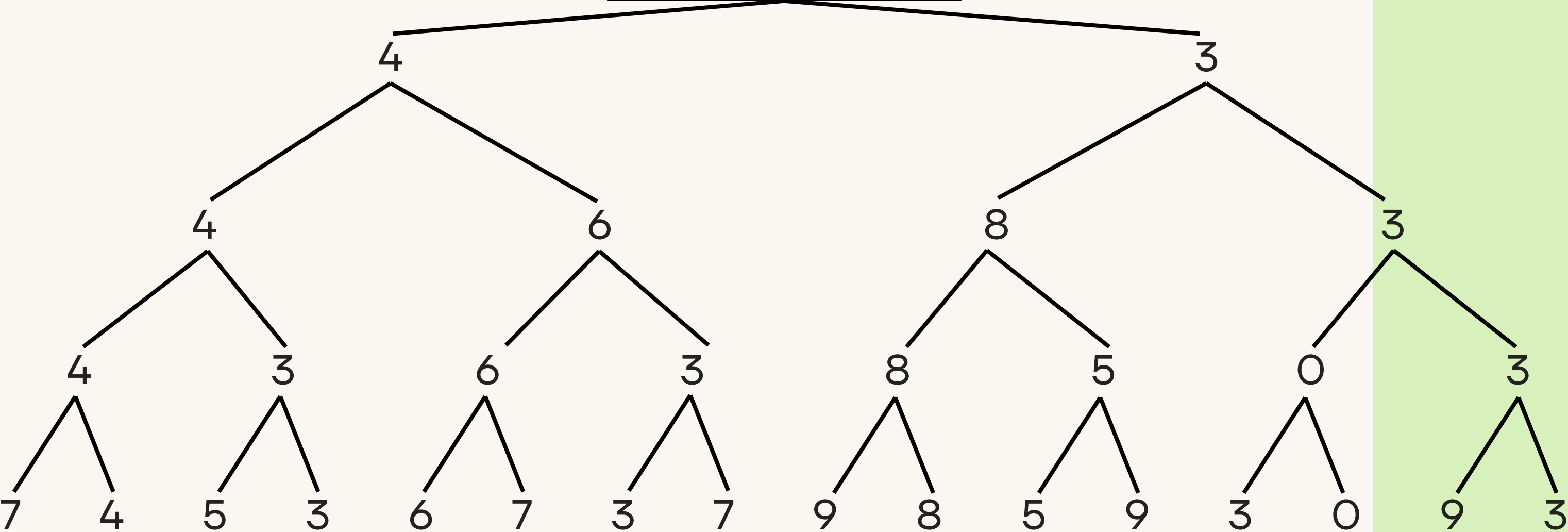
TRY IT!

Handout - 4 Grid Knockout Games

TRY IT!

7	5	9	5
4	3	8	9
6	3	3	9
7	7	0	3

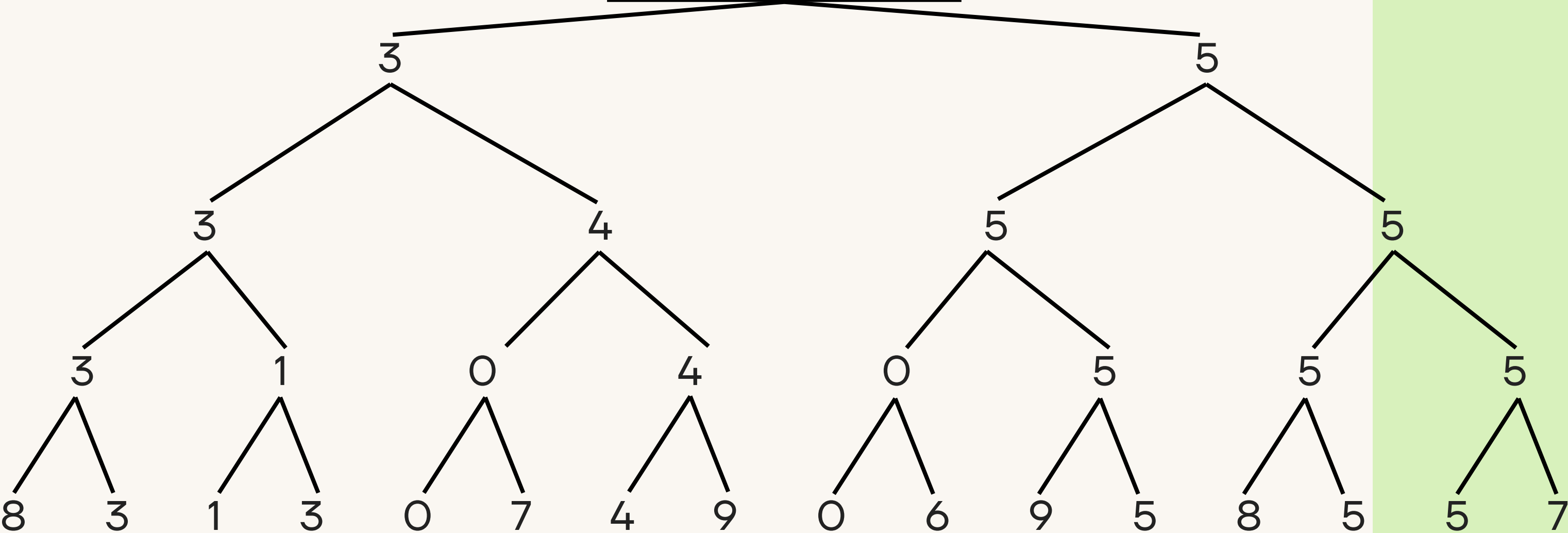
OPTIMAL MOVE: L
 $V = 4$



TRY IT!

8	1	0	9
3	3	6	5
0	4	8	5
7	9	5	7

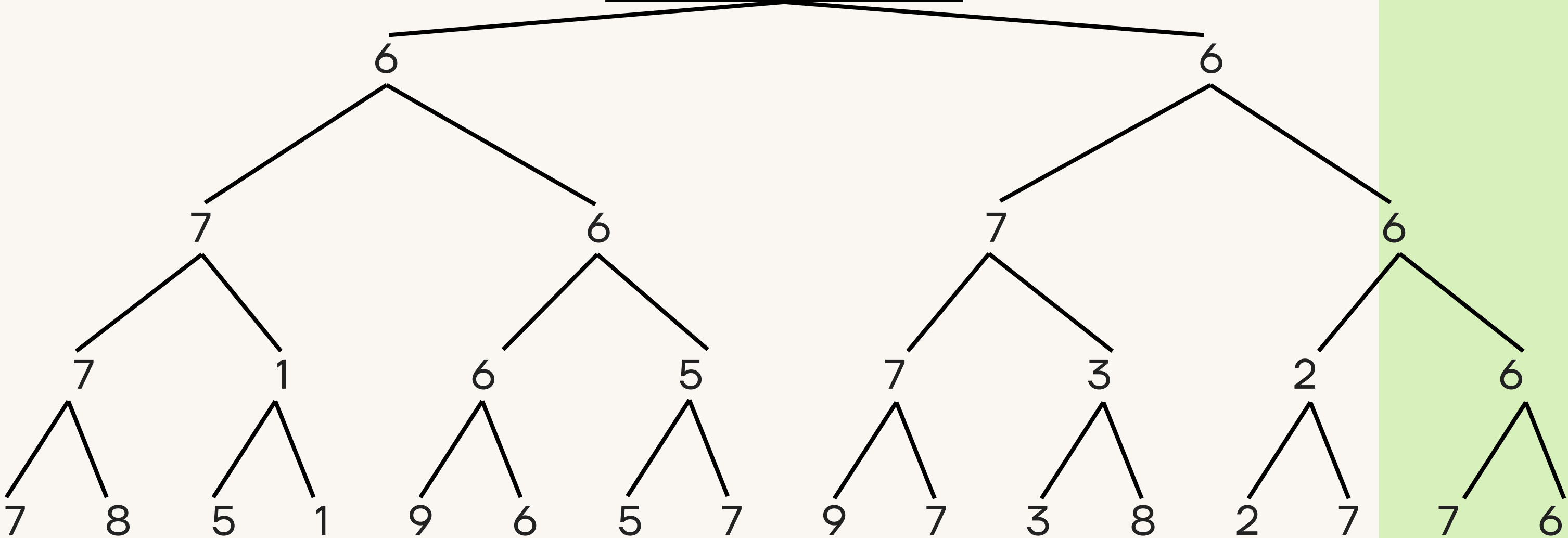
OPTIMAL MOVE: R
 $V = 5$



TRY IT!

7	5	9	3
8	1	7	8
9	5	2	7
6	7	7	6

OPTIMAL MOVE: EITHER
 $V = 6$



CODE IT!

Class Activities → `Minimax.ipynb`

EFFICIENCY

The number of game states is exponential in the depth of the search tree.

Minimax is an exhaustive search since it evaluates all branches and possible terminal states.

Time: $O(b^m)$

Space: $O(bm)$

Can we make it better?

ALPHA-BETA PRUNING

Pruning - eliminating possibilities from consideration *without* having to examine them

Consider a node n somewhere in the tree such that a player has a choice of moving to n . If the Player has a better choice either at the same level or at any point higher up in the tree, then the Player will never move to n .

α = the value of the best choice for MAX

β = the value of the best choice for MIN

ALPHA-BETA PRUNING

function ALPHA-BETA-SEARCH(*game*, *state*) **returns** an action

$\text{player} \leftarrow \text{game}.\text{TO-MOVE}(\text{state})$

$\text{value}, \text{move} \leftarrow \text{MAX-VALUE}(\text{game}, \text{state}, -\infty, +\infty)$

return *move*

function MAX-VALUE(*game*, *state*, α , β) **returns** a (*utility*, *move*) pair

if *game*.IS-TERMINAL(*state*) **then return** *game*.UTILITY(*state*, *player*), *null*

$v \leftarrow -\infty$

for each *a* **in** *game*.ACTIONS(*state*) **do**

$v2, a2 \leftarrow \text{MIN-VALUE}(\text{game}, \text{game}.\text{RESULT}(\text{state}, a), \alpha, \beta)$

if $v2 > v$ **then**

$v, \text{move} \leftarrow v2, a$

$\alpha \leftarrow \text{MAX}(\alpha, v)$

if $v \geq \beta$ **then return** *v*, *move*

return *v*, *move*

$$\begin{aligned} \alpha &= -\cancel{0}9 \\ \nu &= -\cancel{0}9 \\ \beta &= \cancel{0}9 \end{aligned}$$
$$\begin{aligned} \alpha &= 9 \\ \beta &= \alpha 7 \\ v &= t \cancel{6} 7 \end{aligned}$$
$$\begin{aligned} \alpha &= -\infty \\ \beta &= 0 \\ \gamma &= 0 \end{aligned}$$
$$\alpha = -\cancel{10}5^9$$
$$\beta = 0^0$$
$$\gamma = -\cancel{10}5^9$$
$$\Delta$$
$$\begin{aligned} \alpha &= -10 \\ \beta &= 9 \\ \gamma &= -10 \end{aligned}$$
$$\alpha = 9$$
$$\beta = \emptyset$$
$$\gamma = \emptyset$$

$$\begin{array}{l} \alpha = -\infty \\ \beta = \emptyset \\ \gamma = \emptyset \end{array} \quad \wedge \quad \begin{array}{l} \alpha = 5 \\ \beta = \emptyset \\ \gamma = \emptyset \end{array}$$

$$\alpha = -0$$
$$\beta = 9$$
$$\gamma = 0$$
$$\Delta 10$$

$\alpha = 9$
 $\beta = 107$
 $\gamma = 107$
 Δ

$$\alpha = 9$$
$$\beta = \cancel{0}5$$
$$\gamma = \cancel{0}5$$

$\alpha = -104_5$
 $\beta = 30$
 $\gamma = -104_5$

$\alpha = -107$
 $\beta = 5$
 $\gamma = -107$

$\alpha = 51_9$
 $\beta = 0$
 $\gamma = -107_9$

$\alpha = 612$
 $\beta = 9$
 $\gamma = -1012$

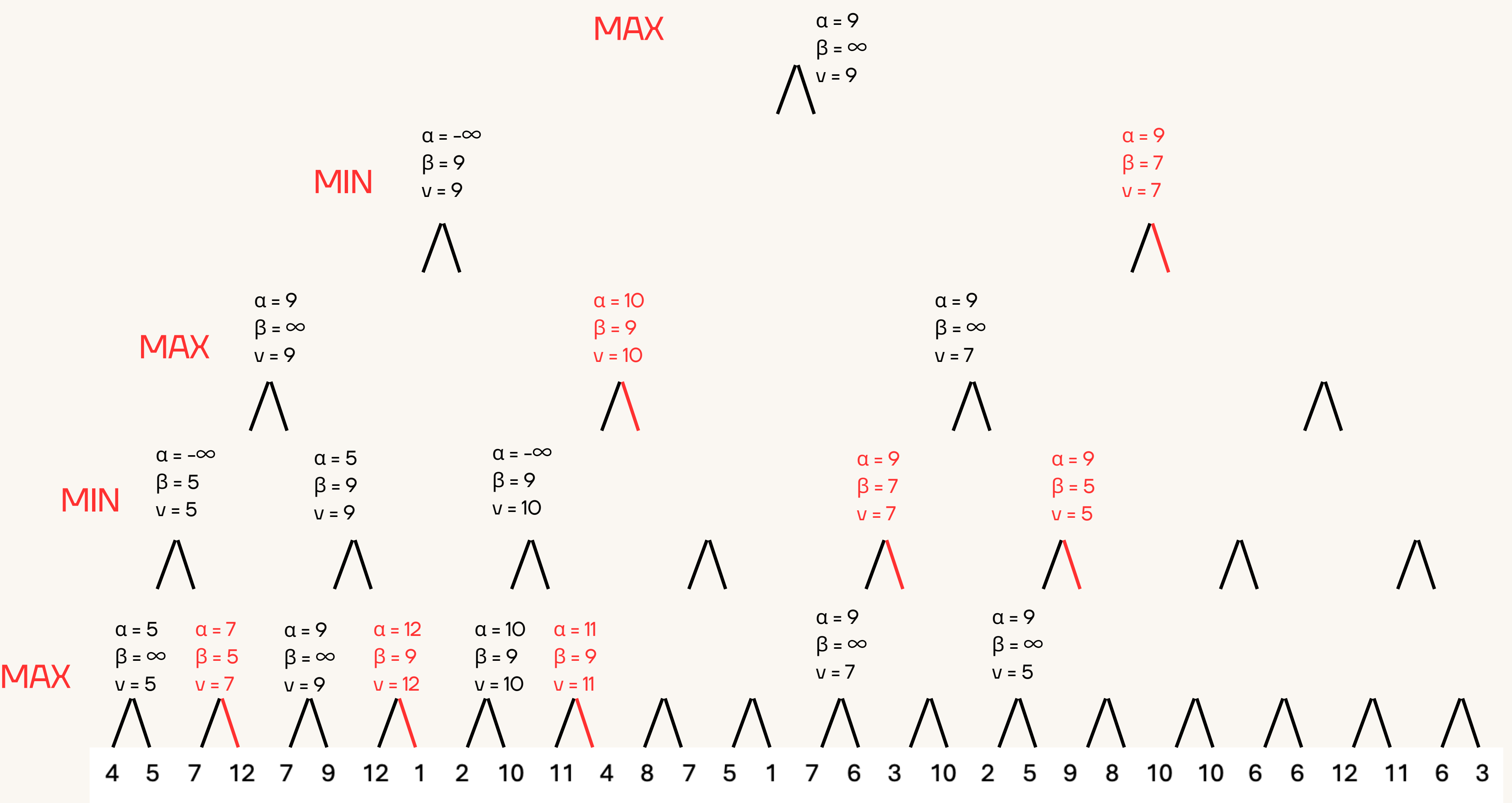
$\alpha = -102$
 $\beta = 910$
 $\gamma = -10210$

$\alpha = -1011$
 $\beta = 9$
 $\gamma = -1011$

$$\begin{aligned}x &= 9 \\ B &= 0 \\ v &= -107\end{aligned}$$
$$\begin{aligned} \alpha &= 9 \\ \beta &= 0 \\ \gamma &= -90 \end{aligned}$$

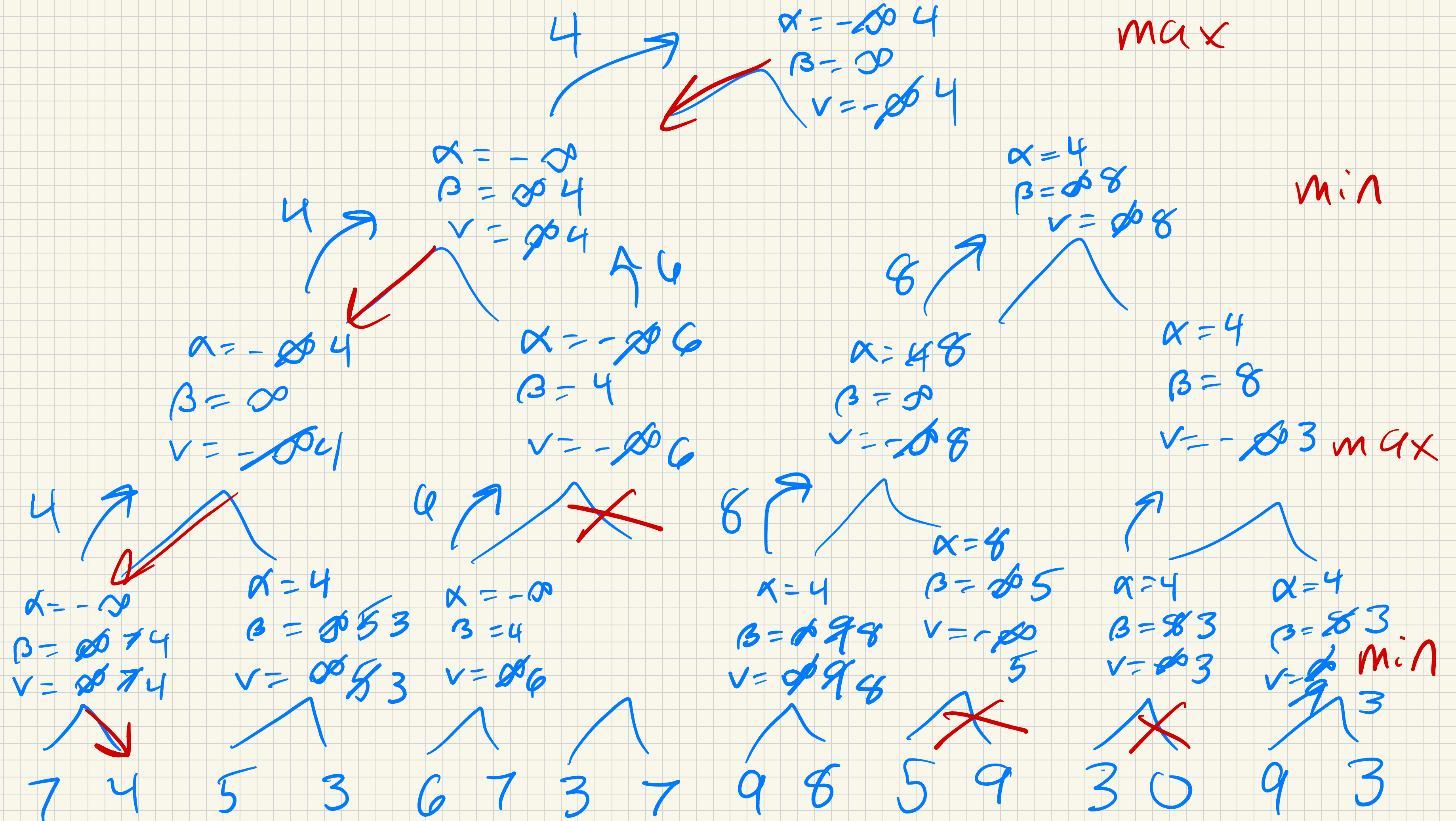
25

4 5 7 12 7 9 12 1 2 10 11 4 8 7 5 1 7 6 3 10 2 5 9 8 10 10 6 6 12 11 6 3



TRY IT!

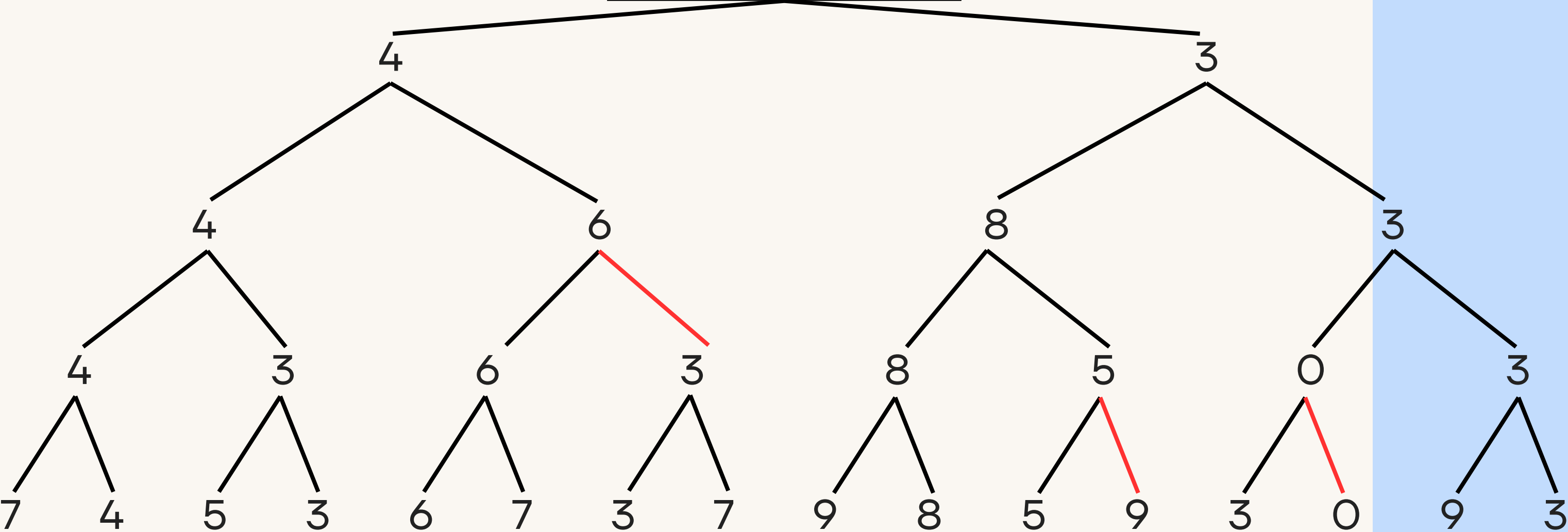
Prune the trees you created from the 4x4 Grid Knockout games.



TRY IT!

7	5	9	5
4	3	8	9
6	3	3	9
7	7	0	3

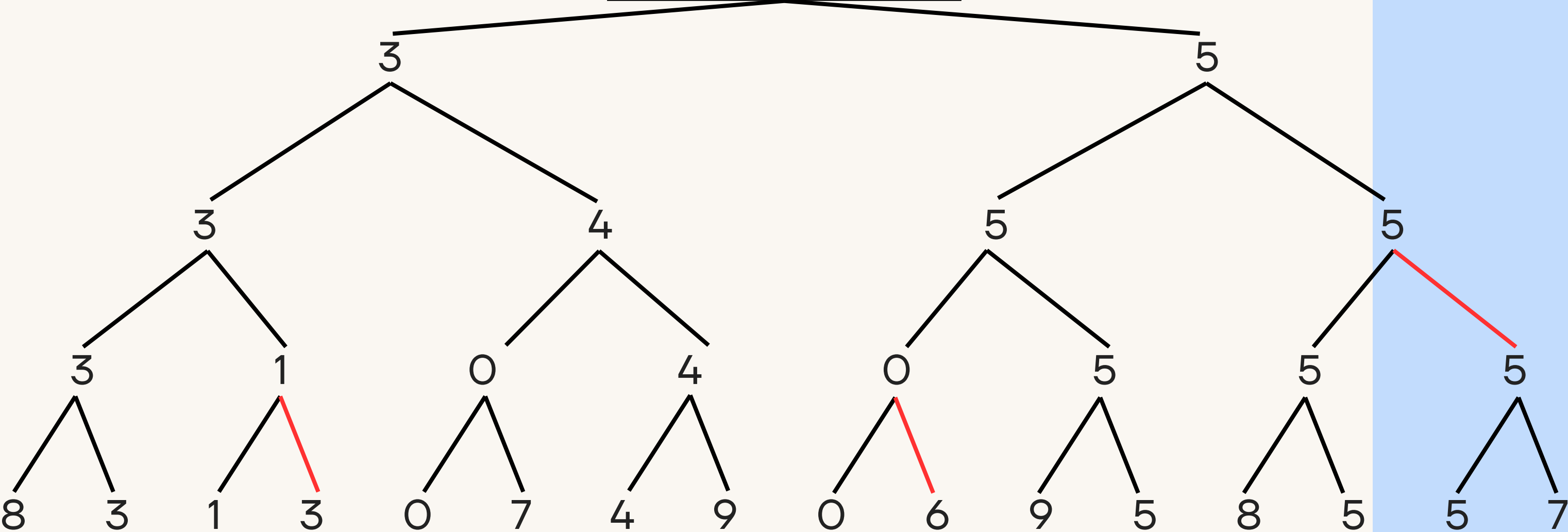
OPTIMAL MOVE: L
 $V = 4$



TRY IT!

8	1	0	9
3	3	6	5
0	4	8	5
7	9	5	7

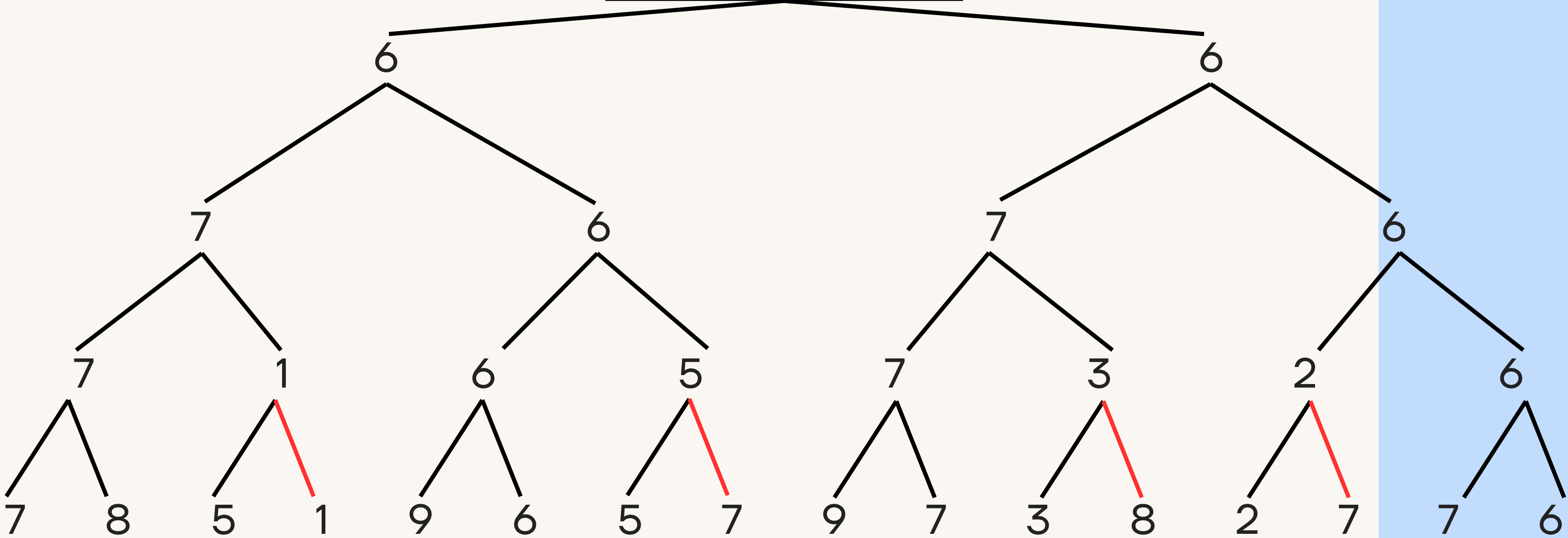
OPTIMAL MOVE: R
 $V = 5$



TRY IT!

7	5	9	3
8	1	7	8
9	5	2	7
6	7	7	6

OPTIMAL MOVE: EITHER
 $V = 6$



RECAP

01

ADVERSARIAL GAMES

Zero-Sum: Players have opposing goals. One player's win is another player's loss

02

MINIMAX

Minimax: Choose moves by assuming the opponent plays optimally

03

A/B PRUNING

Alpha-Beta: Prune subtrees that can't affect the final outcome